

Journal de validation du prototype chatbot — Résultats bruts et discussion

Document de travail indépendant, séparé des chapitres rédigés. Contient l'intégralité des résultats bruts obtenus en suivant `protocole_validation_chatbot.md`, les incidents techniques rencontrés et leur résolution, et les observations qui alimentent le chapitre 3 (section 8) et le chapitre 4 (limites).

Appareil : Galaxy S26 Ultra

Modèle testé : Llama 3.2 1B Q4_K_M **Backend** : `llama-cli` natif en sous-processus (pas `llama-cpp-python` — voir incidents techniques ci-dessous) **Environnement** : Termux, Python 3.14 **Choix de cet appareil** : voir justification détaillée en `chapitre_3_prototype.md` section 8.0

Incidents techniques rencontrés et résolus (dans l'ordre chronologique)

Cette section documente le vrai déroulé de l'installation, avec les blocages réels — utile pour la reproductibilité et pour la section "limites pratiques" du mémoire.

1. **pandas fait échouer l'installation.** Listé dans `requirements.txt` mais jamais réellement importé dans `chatbot.py`, `benchmark.py` ou `utils.py`. Corrigé en commentant la ligne dans `requirements.txt`.
2. **numpy (dépendance de llama-cpp-python) échoue à compiler sous Termux.** Erreur : fonctions `complex`, `long double` (`ccosl`, `csinl`, etc.) non déclarées dans la libc Android (Bionic). Résolu en installant la version précompilée Termux : `pkg install python-numpy` (au lieu de laisser pip compiler depuis les sources).
3. **llama-cpp-python : RuntimeError: Unsupported platform.** Python 3.14 rapporte `sys.platform == "android"` (changement récent de CPython), valeur non reconnue par `llama_cpp/_ctypes_extensions.py` qui ne gère que `linux`, `freebsd`, `darwin`, `win32`, `emscripten`. Patché manuellement sur l'appareil (une ligne, via `sed`) pour ajouter la branche `android`. **Note** : ce correctif n'est finalement pas critique pour le fonctionnement du prototype — `chatbot.py` pilote directement le binaire `llama-cli` compilé au chapitre 2 plutôt que les bindings Python, précisément à cause de ce problème de compatibilité récurrent et documenté dans le code source lui-même.
4. **psutil.cpu_percent() lève PermissionError sur /proc/stat.** Restriction système Android (non-root) qui bloque la lecture des statistiques CPU globales. Corrigé en ajoutant une fonction `safe_cpu_percent()` dans `utils.py` (try/except, retourne 0.0 avec avertissement affiché une seule fois), utilisée par `chatbot.py` et `benchmark.py`.

5. **save_metrics() ne capture pas les métriques réelles par échange en mode interactif.** Bug identifié mais **non corrigé** : results/metrics.json ne contient que des entrées aberrantes générées à la fermeture de session (/exit//quit), avec generated_tokens=1 et des temps de decode absurdes (127s puis 62 810s = 17,4h, mesurant en réalité le temps d'attente écoulé, pas une génération). Aucune des ~20 vraies interactions (12 Q/R + 6 classifications + 3 résumés) n'a été journalisée automatiquement. **Voir tableau comparatif attendu vs résultat plus bas.**
6. **Saturation du contexte.** Après 12 échanges Q/R + 1 tentative de résumé dans la même session (~13 échanges), erreur : request (2226 tokens) exceeds the available context size (2048 tokens), try increasing it. Confirme empiriquement la limite déjà documentée en chapitre 3 section 8.3 ("~10 échanges avant saturation").

Bug save_metrics() — attendu vs résultat observé

	Attendu	Résultat observé
Nombre d'entrées JSON	~20 (une par échange réel)	2 seulement
Déclenchement	Après chaque échange	Une seule fois, à la fermeture de session
generated_tokens	Dizaines par échange	1 dans les deux entrées
decode_time_s	1 à 3s (cohérent avec l'écran)	127,67s puis 62 810s (17,4h)
RAM (ram_before/after/delta_mb)	~770 Mo+	0.0 Mo partout

Partie A — Questions Q/R (12 questions, 2 runs complets)**A.1 — Factuelles simples**

#	Question	Run 1	Verdict	Run 2	Verdict
1	Capitale de la France	“La capitale de la France est Paris.”	✓ Correcte	“La capitale de la France est Paris.”	✓ Correcte
2	Continents sur Terre	“Il existe 7 continents : Afrique, Asie, Amérique, Australie, Europe, Océanie et Amérique du Sud.”	△ Partielle (bon total, liste incohérente — double comptage Amérique/Amérique du Sud, Australie/Océanie)	“Trois continents sont reconnus : l’Afrique, l’Asie et l’Amérique du Nord.”	✗ Incorrecte
3	Symbole chimique de l’eau	“L’eau est représentée par la symbole H2O.”	✓ Correcte	“Le symbole chimique de l’eau est H2O.”	✓ Correcte
4	Année Révolution française	“La Révolution française a eu lieu en 1789.”	✓ Correcte	“La Révolution française a eu lieu en 1789.”	✓ Correcte
5	Auteur “Le Petit Prince”	“Antoine de Saint-Exupéry.”	✓ Correcte	“Antoine de Saint-Exupéry.”	✓ Correcte
6	Plus grande planète	“Jupiter.”	✓ Correcte	“Jupiter.”	✓ Correcte
7	Jours année bissextile	“Un an bissextile a 365,5 jours.”	✗ Incorrecte (attendu : 366)	“365,24 jours” puis “366,2421905 jours” (deux réponses différentes)	✗ Incorrecte

Bilan A.1 : Run 1 = 5✓/1△/1✗ — Run 2 = 5✓/0△/2✗. 5 des 7 questions stables entre les deux runs ; questions 2 et 7 varient à chaque tentative, toujours en échec.

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Run 1	Verdict	Run 2	Verdict
8	Croisement de trains	Méthode fausse (calcule le temps de trajet de chaque train séparément puis soustrait des “retards”), résultat 3,6h (attendu $\approx 1,71$ h)	✗ Incorrecte	Identifie correctement la vitesse combinée ($160+140=300$ km/h) mais applique 512/160 au lieu de 512/300 au moment du calcul → 3,15h	⚠ Partielle (concept correct, exécution fausse)
9	RAM FP16→Q4	Calcule d’abord correctement $14/4=3,5$ Go puis se contredit et conclut à 12 Go	⚠ Partielle	Invente une formule fictive (“ $Q4=Q/4^3$ ”), calcul circulaire, conclut 14 Go pour le FP16 (ne répond pas à la question posée)	✗ Incorrecte
10	Pourquoi decode < prefill en vitesse	Vague, n’explique pas le mécanisme memory-bound/compute-bound	✗ Incorrecte	Confond “prefill” (étape d’inférence) avec “Pre-Training” (entraînement) — erreur catégorielle	✗ Incorrecte
11	RAM smartphone (calcul)	Erreurs arithmétiques en cascade ($5-1,5=“6,5”$ au lieu de 3,5 ; 500 Mo converti en “0,001 Go” au lieu de 0,5) → conclut 6,5 Go restants (attendu 3 Go)	✗ Incorrecte	Déforme les données de l’énoncé elles-mêmes (invente “1 Go pour l’OS” au lieu des 3 Go donnés) → conclut “1 Go restant”	✗ Incorrecte
12	Hybride vs local	Vague ; attribue à tort la “sécurité” à l’hybride, contredisant le cadre du chapitre 1 (section 5.2) qui attribue la confidentialité maximale au 100% local	✗ Incorrecte	Vague et auto-contradictoire (“réduction des coûts” puis “coût de l’infrastructure élevé” dans la même phrase)	✗ Incorrecte

Bilan A.2 : Run 1 = 0✓/1⚠/4✗ — Run 2 = 0✓/1⚠/4✗. Score identique, mais **aucune question n’échoue de la même façon d’un run à l’autre** — le raisonnement n’est pas seulement faible, il est instable dans sa manière d’être faux.

A.2 bis — Run 3 (post-correctif `save_metrics()` / `generate_response()`) Exécuté après la correction du bug documenté à l’incident technique #5 ci-dessus, ainsi que des problèmes de sous-processus llama-cli découverts pendant les tests de validation du correctif sur cet appareil (mode conversation forcé par le chat template, sortie écrite directement sur le TTY plutôt que sur stdout/stderr redirigés — voir chapitre 3/4 pour le détail). Contrairement aux runs 1 et 2, chaque échange de cette session a

été correctement journalisé automatiquement dans `results/metrics.json`, sans aucune anomalie de type `decode_time_s` aberrant.

#	Question (résumée)	Run 3	Verdict	prompt tokens	generated tokens	prefill_time_s	decode_time_s	total_time_s	reload≈
8	Croisement de trains	Identifie la vitesse combinée ($160+140=300$ km/h) et calcule $512/300 = 1,68$ h (≈ 100 min)	✓ Correcte (écart d'arrondi mineur avec l'attendu 1,71h)	156	134	0,799s	2,538s	4,497s	1,160s
9	RAM FP16→Q4	Invente une troisième formule différente ("taille du modèle / taille du processeur × 8"), ignore le 14 Go donné dans l'énoncé, invente "16 Go", conclut 8 Go	✗ Incorrecte	333	108	1,998s	2,123s	5,301s	1,180s
10	Pourquoi decode < prefill en vitesse	Troisième explication différente, toujours sans mentionner le mécanisme memory-bound/compute-bound	✗ Incorrecte	494	97	3,153s	2,026s	6,351s	1,172s
11	RAM smartphone (calcul)	N'effectue aucun calcul : "Il n'y a pas d'informations supplémentaires à fournir pour justifier le reste de la réponse."	✗ Incorrecte (refus déguisé, différent des erreurs arithmétiques des runs 1-2)	682	25	4,951s	0,515s	6,651s	1,185s
12	Hybride vs local	Réponse hors sujet sous forme de liste à puces, ne compare pas edge/cloud vs local comme demandé	✗ Incorrecte	768	73	6,057s	2,200s	9,469s	1,212s

Bilan A.2 Run 3 : 1✓/0△/4✗ — cohérent avec les runs 1 et 2 (0✓/1△/4✗ chacun) : le raisonnement multi-étapes reste en échec sur 4 des 5 questions, avec, une fois de plus, un mode d'échec différent à chaque tentative (troisième formule inventée pour Q9, troisième explication bancal pour Q10, refus pur plutôt que calcul erroné pour Q11 et Q12). Seule différence notable sur les trois runs : Q8 obtient ici une réponse quasi correcte (méthode ET résultat numérique proches de l'attendu), suggérant que le modèle est capable de résoudre ce type de problème, mais de façon non fiable plutôt que jamais.

Temps de rechargement (`total_time_s - (prefill_time_s + decode_time_s)`) sur ces 5 échanges : 1,16 à 1,21s, cohérent avec la mesure indépendante de $1,175s \pm 0,005s$ obtenue sur un autre échantillon de 5 tours (voir mesure complémentaire dédiée au

temps de rechargement) — confirme un coût fixe par tour, indépendant du contenu de la question et de la taille croissante du contexte.

Bilan Partie A (12 questions, runs 1 et 2 seulement — A.1 non retestée au run 3) : Run 1 = 5✓/2△/5✗ — Run 2 = 5✓/1△/6✗.

Partie B — Résumé de texte (3 runs, même texte source)

Texte source : voir `protocole_validation_chatbot.md`, Partie B (paragraphe sur les trois piliers de l’inférence LLM embarquée).

Point clé attendu	Run 1	Run 2	Run 3
Quantification ($\div 4$ - $\div 8$, perte qualité limitée)	✓ Présent, complet	✓ Présent, complet	△ Partiel (facteur et perte qualité non chiffrés)
Frameworks (ARM NEON / NPU)	✓ Présent, complet	✓ Présent, complet (séparé en 2 points)	✗ Absent (juste un titre générique, sans ARM NEON ni NPU)
Gestion mémoire (memory-mapping)	△ Partiel (mémoire mentionnée, mmap absent)	✓ Présent, complet (cite “mmap” explicitement)	△ Partiel (contrainte RAM mentionnée, mmap absent)
Conclusion (viabilité 1-3B depuis 2020)	✓ Présent, quasi mot pour mot	✗ Absent	✓ Présent, quasi mot pour mot
Verdict global	Bon (3 pts complets)	Bon (3 pts complets)	Partiel (1 pt complet, 2 partiels, 1 absent)

Note méthodologique : le premier essai du run 1 (via /résumé en pleine conversation, après les 12 questions) a été invalidé par la saturation du contexte — voir incident technique #6. Les 3 runs retenus ci-dessus ont tous été exécutés en process séparé (--task summary, contexte vide).

Conclusion : aucun des 4 points clés n’est présent dans les 3 runs simultanément, sauf approximativement la conclusion finale (2/3) et la quantification (2,5/3). Le point le plus instable est “frameworks NEON/NPU” (présent-présent-absent) — preuve chiffrée que la qualité du résumé varie significativement d’un passage à l’autre pour un texte et un modèle identiques.

Partie C — Classification de sentiment (2 runs, 6 textes)

#	Texte (résumé)	Attendu	Run 1	Run 2
1	Smartphone excellent	POSITIF	[POSITIF] 95% ✓	[POSITIF] 95% ✓
2	Écran cassé, déçu	NÉGATIF	[NÉGATIF] 75% ✓	[NÉGATIF] 90% ✓ (justification tronquée en fin)
3	Fonctionne, rien à signaler	NEUTRE	[NEUTRE] 80% ✓	[NEUTRE] 85% ✓
4	Livreur à l'heure, colis en bon état	POSITIF	[POSITIF] 90% ✓	[POSITIF] 98% ✓
5	"Service rapide"... 3 semaines (ironie)	NÉGATIF	[NÉGATIF] 65% ✓	[NÉGATIF] 72% ✓ — justification mentionne explicitement un "ton sarcastique"
6	Correspond à la description, sans plus	NEUTRE	[NEUTRE] 85% ✓	[NEUTRE] 80% ✓

Bilan Partie C : 6/6 sur les deux runs (12/12 au total), y compris le cas piège d'ironie (texte #5).

Observation importante à nuancer dans la rédaction : l'hypothèse initiale du mémoire ("nuances et ironie mal détectées", chapitre 3 section 8.3 / chapitre 4 section 1.1) n'est **pas confirmée** par ces tests — le modèle classe correctement le texte ironique dans les deux runs, avec une confiance plus basse que la moyenne (65-72% vs 75-98% sur les autres textes) mais un label juste. Au run 2, la justification nomme explicitement le "ton sarcastique", suggérant une détection au moins partielle du registre ironique plutôt qu'une simple corrélation lexicale sur "trois semaines d'attente".

Défaut technique observé : justifications tronquées en fin de phrase sur les textes #2 et #5 au run 2 — probablement une limite de tokens de génération (max_tokens) trop courte pour ce mode.

Partie D — Observations de session

Mesure	Valeur observée	Fiabilité
Temps de démarrage du script (hors chargement modèle)	0,245s (time, mode --mock)	Fiable, mais ne mesure pas le chargement réel du modèle
Chargement du modèle réel (par tour, sous-processus llama-cli)	Qualitatif : “quelques secondes” (mention explicite dans l’app elle-même)	Non chronométré précisément — limite assumée
RAM disponible au repos (post-session)	3 935 Mo disponibles / 11 122 Mo total	Mesuré après la fin de session, pas pendant l’inférence active
Swap utilisé	4 907 Mo / 12 287 Mo	Swap actif malgré un modèle de seulement ~770 Mo — à surveiller, possible signe de pression mémoire générale du système, pas spécifiquement liée au modèle
Nombre d’échanges avant saturation du contexte	~13 (12 Q/R + 1 tentative de résumé)	Mesure directe, fiable — confirme la limite documentée en chapitre 3
Comportement à la saturation	Erreur explicite et bloquante (request (2226 tokens) exceeds the available context size (2048 tokens)), pas de dégradation silencieuse	Observation fiable

Synthèse — Galaxy S26 Ultra

Partie	Résultat
A.1 Factuelles (7)	Run 1 : 5✓/1△/1✗ — Run 2 : 5✓/0△/2✗
A.2 Raisonnement (5)	Run 1 : 0✓/1△/4✗ — Run 2 : 0✓/1△/4✗ — Run 3 (post-correctif) : 1✓/0△/4✗
B. Résumé	Run 1 : Bon — Run 2 : Bon — Run 3 : Partiel
C. Classification (6)	Run 1 : 6/6 — Run 2 : 6/6
Saturation contexte	Confirmée après ~13 échanges
Bugs identifiés	pandas (corrigé), numpy/Termux (corrigé), llama-cpp-python Android (patché, non critique), psutil.cpu_percent (corrigé), save_metrics() (corrigé — voir A.2 bis, Run 3)

Points forts confirmés et stables : classification de sentiment (12/12 sur 2 runs, y compris ironie), questions factuelles courantes (Paris, H2O, 1789, Saint-Exupéry, Jupiter — 4/4 stables sur les 2 runs).

Points faibles confirmés et stables : raisonnement multi-étapes (1/5 sur les runs 1 et 2, 1/5 également au run 3 en comptant la question correcte), avec des modes

d'échec différents à chaque tentative — le modèle n'est pas seulement mauvais en raisonnement, il est imprévisible dans sa façon de l'être.

Limite méthodologique assumée pour les runs 1 et 2 uniquement : la journalisation automatique des métriques (`save_metrics()`) ne fonctionnait pas correctement en mode interactif au moment de ces deux runs — les données de ces deux runs reposent sur la lecture manuelle des sorties écran ([Prompt: X t/s | Generation: Y t/s]), pas sur les fichiers JSON générés par le prototype. **Ce bug est désormais corrigé** (voir A.2 bis, Run 3, ci-dessus) : la cause racine combinait un bug de parsing des statistiques de `llama-cli` dans `save_metrics()/generate_response()` et un problème distinct de sous-processus (absence de `stdin` explicite, mode conversation forcé par le chat template, sortie TTY non redirigée) qui empêchait le sous-processus de se terminer proprement à chaque tour en mode interactif. Le run 3 ci-dessus démontre que la journalisation automatique fonctionne désormais correctement, avec des métriques plausibles sur les 5 échanges.

Appareil : Galaxy A16

Modèle testé : Llama 3.2 1B Q4_K_M (identique au S26 Ultra, pour comparabilité directe) **Backend** : `llama-cli` natif en sous-processus **Environnement** : Termux, Python 3.14 **SoC** : Exynos 1330 — représentant entrée de gamme du corpus (~200 €), à l'opposé du Snapdragon 8 Elite du S26 Ultra

Incidents techniques rencontrés et résolus

Le repo cloné sur l'A16 ne contenait pas encore les correctifs poussés depuis le poste de développement (le push GitHub Desktop n'avait pas été fait avant le début de ce test). Les mêmes bugs déjà résolus sur le S26 Ultra se sont donc reproduits à l'identique et ont dû être corrigés **manuellement sur l'appareil** :

1. **git pull initial en échec** : TLS connect error transitoire lors de la mise à jour du dépôt `llama.cpp` — résolu par une simple nouvelle tentative (aléa réseau, pas un bug du projet).
2. **psutil refuse de compiler** : `platform android is not supported` — nouvelle variante du même problème que `numpy/llama-cpp-python` (Python 3.14 rapporte `sys.platform == "android"`), mais cette fois le blocage vient du script de build de `psutil` lui-même (version 7.2.2, plus récente que celle utilisée sur le S26 Ultra, qui a ajouté cette vérification explicite). Résolu via `pkg install python-psutil` (paquet Termux précompilé) plutôt que la compilation source.
3. **pandas fait échouer l'installation** : le correctif (retrait de `pandas` de `requirements.txt`) n'étant pas encore poussé sur GitHub, réappliqué manuellement sur l'appareil via `sed`.

4. **numpy échoue à compiler** : même cause que sur le S26 Ultra (fonctions `complex.h` absentes de la lib `Bionic`), réglé via `pkg install python-numpy`.
5. **llama-cpp-python : RuntimeError: Unsupported platform** : identique au S26 Ultra, sans conséquence puisque `chatbot.py` ne dépend pas de ce binding.
6. **psutil.cpu_percent() → PermissionError sur /proc/stat** : le correctif `safe_cpu_percent()` n'étant pas non plus poussé, réappliqué manuellement (ajout de la fonction dans `utils.py` + remplacement des appels dans `chatbot.py` via `sed`).
7. **save_metrics() reproduit le même artefact de fin de session** que sur le S26 Ultra (1 seul token "généré", ~200s de decode, 0 Mo RAM, 0% CPU à la fermeture) — confirme que ce bug n'est pas spécifique à un appareil, mais bien un défaut du code lui-même.

Enseignement méthodologique : l'ensemble de ces répétitions aurait été évité si les correctifs avaient été poussés sur GitHub avant de démarrer les tests sur le second appareil — à faire systématiquement avant tout nouveau test sur un appareil supplémentaire.

Partie A — Questions Q/R (12 questions, 1 run)

A.1 — Factuelles simples

#	Question	Réponse A16	Verdict
1	Capitale de la France	"La capitale de la France est Paris."	✓ Correcte
2	Continents sur Terre	"Il y a trois continents : Terre, Mer et Atmosphère."	✗ Incorrecte (réponse incohérente — ce ne sont pas des continents)
3	Symbole chimique de l'eau	"Le symbole chimique de l'eau est H2O."	✓ Correcte
4	Année Révolution française	"La Révolution française a eu lieu en 1789."	✓ Correcte
5	Auteur "Le Petit Prince"	"Antoine de Saint-Exupéry."	✓ Correcte
6	Plus grande planète	"Jupiter."	✓ Correcte
7	Jours année bissextile	"365,24 jours"	✗ Incorrecte (attendu 366 ; même erreur numérique que le run 2 du S26 Ultra)

Bilan A.1 : 5✓ / 0⚠ / 2✗ — identique en score aux deux runs du S26 Ultra (mêmes questions 2 et 7 en échec).

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Réponse A16	Verdict
8	Croisement de trains	Erreurs de calcul en cascade (temps de trajet mal recalculé), conclut à une “distance de 0 km” et que les trains “se croisent exactement à Paris ET à Lyon” — logiquement impossible	✗ Incorrecte
9	RAM FP16→Q4	Passe par une valeur intermédiaire correcte (3,5) mais en mauvaise unité (“millions de paramètres” au lieu de Go), puis continue à diviser par 4 une seconde fois → “875 millions de paramètres”, ne répond pas à la question posée	✗ Incorrecte
10	Pourquoi decode < prefill en vitesse	Vague et circulaire, ne mentionne ni le traitement parallèle du prompt ni la contrainte de bande passante mémoire	✗ Incorrecte
11	RAM smartphone (calcul)	Confond 800 Mo avec “800 Go” dans la formule affichée ; résultat final “3 Go” incohérent avec le calcul écrit (6–2–1–800 ne donne pas 3)	✗ Incorrecte
12	Hybride vs local	Hors-sujet : parle de frameworks de développement mobile générique (React Native, Flutter, jeux, réalité virtuelle) sans jamais traiter la question du compromis local/cloud pour l’inférence LLM	✗ Incorrecte

Bilan A.2 : 0✓ / 0⚠ / 5✗ — échec total, alors que chaque run du S26 Ultra comptait au moins 1 réponse partielle.

Bilan Partie A (12 questions) : 5✓ / 0⚠ / 7✗.

Réserve méthodologique importante : ce score plus faible en raisonnement sur l’A16 ne doit pas être interprété comme un effet du matériel. Le modèle, la quantification (Q4_K_M) et les paramètres de génération (temperature=0.7, non nul) sont strictement identiques à ceux utilisés sur le S26 Ultra — la sortie du modèle n’est pas déterministe d’un run à l’autre, indépendamment de l’appareil. Ce run doit être lu comme un **3ème échantillon indépendant** de la distribution de réponses du modèle sur les questions de raisonnement, pas comme une preuve que l’A16 “raisonne moins bien”. Ce qui reste, en revanche, directement attribuable au matériel : la vitesse (voir ci-dessous).

Partie B — Résumé de texte (1 run, même texte source que le S26 Ultra)

Point clé attendu	Résultat A16
Quantification ($\div 4$ - $\div 8$, perte qualité limitée)	✓ Complet — “divisant la taille du modèle par 4 à 8 avec une perte de qualité limitée à quelques points de pourcentage”
Frameworks (ARM NEON / NPU)	⚠ Partiel — mentionne ARM NEON, mais l’attribue à tort à la fois à llama.cpp ET à ML Kit GenAI (le texte source les oppose : NEON pour llama.cpp, NPU pour ML Kit GenAI) ; le NPU n’est jamais cité
Gestion mémoire (memory-mapping)	⚠ Partiel — décrit le principe correctement, mais le terme “mmap”/“memory-mapping” n’apparaît jamais, comme sur les runs du S26 Ultra
Conclusion (viabilité 1-3B depuis 2020)	✓ Complet — quasi mot pour mot

Verdict : Bon — cohérent avec les runs 1 et 2 du S26 Ultra (2 points complets, 2 partiels).

Partie C — Classification de sentiment (1 run, 6 textes)

#	Texte (résumé)	Attendu	Résultat A16
1	Smartphone excellent	POSITIF	[POSITIF] 95% ✓
2	Écran cassé, déçu	NÉGATIF	[NÉGATIF] 20% ✓ — label correct mais justification interne incohérente (“ce qui rend la réaction générale neutre”, contredisant le label NÉGATIF produit)
3	Fonctionne, rien à signaler	NEUTRE	[NEUTRE] 90% ✓
4	Livreur à l’heure, colis en bon état	POSITIF	[POSITIF] 74% ✓
5	“Service rapide”... 3 semaines (ironie)	NÉGATIF	[NEUTRE] 2% ✗ — échec du cas piège ; contradiction interne (la justification dit “attitude négative envers le service” mais le label produit est NEUTRE, avec une confiance de seulement 2%)
6	Correspond à la description, sans plus	NEUTRE	[NÉGATIF] 90% ✗ — justification incohérente, mentionne un “outil informatique” sans lien avec le texte

Bilan Partie C : 4/6 — contre 6/6 sur les deux runs du S26 Ultra. Les deux échecs tombent précisément sur les deux cas les plus subtils du protocole (l’ironie du texte 5, le sentiment neutre/mitigé du texte 6).

Réserve à intégrer dans la rédaction : la conclusion provisoire tirée des deux runs du S26 Ultra (“l’ironie est correctement détectée, contredisant l’hypothèse initiale du mémoire”) doit être **nuancée** à la lumière de ce 3ème échantillon. Sur 3 tentatives indépendantes du même cas piège, le résultat est 2/3 correct, pas 3/3 — la détection de l’ironie apparaît **instable** plutôt que fiable. Le chapitre 3/4 ne doit pas présenter la

détection de l'ironie comme acquise, mais comme un résultat mitigé et dépendant de l'échantillon.

Partie D — Observations de session

Mesure	Valeur observée	Comparaison S26 Ultra
Temps de démarrage du script (hors chargement modèle, mode --mock)	1,060 s	0,245 s — l'A16 est ~4× plus lent même pour le seul démarrage Python, cohérent avec l'écart de puissance CPU brute
RAM disponible au repos (post-session)	1 950 Mo disponibles / 5 452 Mo total ; 1 175 Mo libres	3 935 Mo disponibles / 11 122 Mo total — l'A16 dispose d'une marge RAM beaucoup plus restreinte
Swap utilisé	2 563 Mo / 8 191 Mo	4 907 Mo / 12 287 Mo
Nombre d'échanges avant saturation du contexte	Non retesté sur cet appareil	Ce paramètre (-c 2048, fixé dans le code) ne dépend pas du matériel mais du nombre de tokens cumulés dans la conversation — la valeur mesurée sur le S26 Ultra (~13 échanges) est réutilisable telle quelle, retester sur l'A16 n'aurait apporté aucune information supplémentaire

Vitesses d'inférence observées sur l'ensemble des tests A16 : prefill entre 12,1 et 52,0 t/s, decode entre 3,9 et 7,8 t/s — contre respectivement ~150-235 t/s et ~50-60 t/s sur le S26 Ultra. Écart net et cohérent avec la différence de segment matériel (Exynos 1330 entrée de gamme vs Snapdragon 8 Elite flagship), confirmant que l'A16 reste néanmoins utilisable en conversationnel malgré une latence par réponse nettement plus élevée.

Synthèse — Galaxy A16

Partie	Résultat
A.1 Factuelles (7)	5✓/0△/2✗
A.2 Raisonnement (5)	0✓/0△/5✗
B. Résumé	Bon (2 points complets, 2 partiels)
C. Classification (6)	4/6 — échecs sur l'ironie (#5) et le neutre mitigé (#6)
Bugs identifiés	Mêmes bugs que le S26 Ultra (pandas, numpy, psutil platform, psutil.cpu_percent, save_metrics()), non encore poussés sur GitHub au moment du test — tous corrigés manuellement sur l'appareil
Vitesse	Prefill 12-52 t/s, decode 4-8 t/s — nettement plus lent que le S26 Ultra, comme attendu pour un appareil entrée de gamme

Point le plus important pour la rédaction : ce run agit comme un 3ème échantillon indépendant du comportement du modèle (mêmes poids, même quantification, même température non nulle) plutôt que comme une preuve d'un effet du matériel sur la qualité des réponses. Il nuance la conclusion optimiste sur la détection de l'ironie tirée des deux runs S26 Ultra (2/3 sur l'échantillon complet, pas 3/3) et confirme un échec quasi total sur le raisonnement multi-étapes (0/5), un poil plus marqué que sur le S26 Ultra (1/5 à chaque run) mais dans la même zone de résultat globalement très faible. Le seul écart clairement attribuable au matériel, sans ambiguïté, est la **vitesse** — prefill et decode nettement inférieurs, cohérent avec l'écart de segment de gamme entre les deux appareils.

Appareil : Infinix Hot 60i 5G

Modèle testé : Llama 3.2 1B Q4_K_M (identique aux deux appareils précédents, pour comparabilité directe) **Backend** : llama-cli natif en sous-processus (build b9-13e6738) **Environnement** : Termux, Python 3.13.13 **SoC** : Dimensity 6400 (MediaTek, 6nm) — troisième vendeur de SoC du corpus après Qualcomm (S26 Ultra) et Samsung Exynos (A16)

Incidents techniques rencontrés et résolus

1. **Repo jamais cloné sur cet appareil avant ce test** (contrairement à l'A16, où il était présent mais désynchronisé) : clonage initial depuis GitHub — llama.cpp était déjà compilé (chapitre 2), mais prototype-cli/ était absent.
2. **pip install psutil échoue à la compilation** : platform android is not supported — même famille de problème que sur l'A16, résolu via pkg install python-psutil (paquet Termux précompilé).
3. **Décalage de version après le pkg install initial** : python-psutil annoncé « already the newest version », mais import psutil échoue quand même (ModuleNotFoundError), avec 43 paquets Termux non mis à jour en attente — signe d'un décalage entre le paquet python système et l'extension C python-psutil compilée pour une version antérieure. Résolu par pkg update && pkg upgrade (mise à jour groupée de tous les paquets Termux, dont python et python-psutil simultanément), confirmé par python -c "import psutil; print(psutil.__version__)" → 7.2.2.
4. **Nouveau, spécifique à cet appareil : incompatibilité de flag llama-cli**. Le build installé (b9-13e6738, plus récent que celui compilé sur le S26 Ultra/A16) refuse le flag --no-conversation utilisé par chatbot.py (--no-conversation is not supported by llama-cli please use llama-completion instead) et retombe sur son propre mode conversation interne. Conséquence observée en mode chat interactif : chaque nouveau sous-processus rejoue l'intégralité de l'historique de

la conversation dans sa réponse au lieu de répondre uniquement à la nouvelle question — comportement absent sur le S26 Ultra et l'A16. Non corrigé au moment de ce test (nécessiterait d'adapter `chatbot.py` pour détecter la version de `llama-cli` et ajuster les flags en conséquence) ; documenté comme limite plutôt que contourné en cours de protocole, pour ne pas invalider la mesure en la modifiant a posteriori.

Partie A — Questions Q/R (12 questions, 1 run)

A.1 — Factuelles simples

#	Question	Réponse Infinix	Verdict
1	Capitale de la France	« La capitale de la France est Paris. »	✓ Correcte
2	Continents sur Terre	« Il y en a 7. »	✓ Correcte
3	Symbole chimique de l'eau	« Tu as besoin d'une réponse plus longue ? »	✗ Incorrecte (aucune réponse donnée à la question posée)
4	Année Révolution française	Réponse noyée dans un rappel de tout l'historique de conversation (incident technique #4), contient « La Révolution française a eu lieu en 1789 »	✓ Correcte
5	Auteur « Le Petit Prince »	Idem, rappel intégral + « L'auteur de "Le Petit Prince" est Antoine de Saint-Exupéry »	✓ Correcte
6	Plus grande planète	« la plus grande planète du système solaire est la Terre, mais tu veux parler de la planète la plus éloignée [...] Jupiter, qui se trouve dans la constellation d'Andromède »	⚠ Partielle (le mot juste « Jupiter » apparaît, mais précédé d'une affirmation fausse — la Terre présentée comme la plus grande planète — et d'un raisonnement confus assimilant « plus grande » à « plus éloignée »)
7	Jours année bissextile	« 365,2425 jours »	✗ Incorrecte (attendu 366 ; même erreur numérique que l'A16 et le run 2 du S26 Ultra)

Bilan A.1 : 4✓ / 1⚠ / 2✗ — légèrement inférieur aux trois runs précédents (5✓ à chaque fois). La question 6, habituellement stable ailleurs, échoue ici pour la première fois, dans un contexte où le rappel intégral de l'historique (incident #4) commence déjà à dégrader la cohérence des réponses.

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Réponse Infinix	Verdict
8	Croisement de trains	Calcule les deux temps de trajet séparément (3,15h et 3,64h) au lieu de la vitesse combinée, puis une soustraction incohérente → conclut « 4 heures » (attendu ≈1,71h)	✗ Incorrecte
9	RAM FP16→Q4	Invente une formule fictive « $(7 \times 7 \times [\text{vide}]) / 8 = 392 \text{ Go}$ » sans utiliser les 14 Go donnés dans l'énoncé, puis divise par 4 → « 98 Go »	✗ Incorrecte
10	Pourquoi decode < prefill en vitesse	Vague, aucune mention de memory-bound/compute-bound ; affirme à tort que le prefill « ne nécessite pas de lecture du contenu »	✗ Incorrecte
11	RAM smartphone (calcul)	Confond tokens et Go (« 2048 tokens [...] nécessitera environ 2048 Go de RAM »), calcul incohérent, conclut « 512 Go » nécessaires	✗ Incorrecte
12	Hybride vs local	Non complétée : saturation du contexte avant génération (request (2117 tokens) exceeds the available context size (2048 tokens)), aucune réponse produite	Non évaluable

Bilan A.2 : 0✓ / 0△ / 4✗ + 1 non évaluable — échec total sur les questions traitées, cohérent avec le S26 Ultra (0-1/5 par run) et l'A16 (0/5).

Bilan Partie A (11 questions traitées sur 12) : 4✓ / 1△ / 6✗ + 1 non évaluable.

Réserve méthodologique importante : contrairement au run A16 (mêmes conditions expérimentales que le S26 Ultra, seule la sortie du modèle diffère par non-déterminisme), ce run Infinix n'est **pas directement comparable** aux deux précédents pour le score factuel et la saturation du contexte : le build llama-cli plus récent (incident #4) modifie le comportement même du programme testé, indépendamment du modèle ou du matériel. Le score de raisonnement multi-étapes (0/5), en revanche, reste comparable et vient renforcer la conclusion déjà établie : c'est la troisième confirmation indépendante, sur un troisième vendeur de SoC (MediaTek, après Qualcomm et Samsung), de l'échec quasi total du modèle sur ce type de tâche.

Partie B — Résumé de texte (1 run, même texte source que les deux autres appareils)

Point clé attendu	Résultat Infinix
Quantification ($\div 4 - \div 8$, perte qualité limitée)	⚠ Partiel — concept mentionné, facteur et perte de qualité non chiffrés
Frameworks (ARM NEON / NPU)	⚠ Partiel — ARM NEON explicitement cité, le NPU/ML Kit GenAI omis
Gestion mémoire (memory-mapping)	✓ Complet — cite explicitement « memory-mapping (mmap) »
Conclusion (viabilité 1-3B depuis 2020)	✓ Complet — quasi mot pour mot

Verdict : Bon (2 points complets, 2 partiels, aucun absent) — la couverture la plus complète des 4 points observée sur les trois appareils testés jusqu’ici.

Défaut de forme observé : répétition dégénérative dans le texte généré (« inférer des informations de contenu du contenu du modèle » répété quasi à l’identique) — artefact distinct de l’incident #4, puisque ce mode tourne en process séparé à contexte vide.

Métriques : 428 tokens prompt, 378 tokens générés, prefill 7,93 s (54,0 tok/s), decode 50,46 s (7,5 tok/s), total 83,98 s, +1714 Mo RAM (pic sous-processus).

Partie C — Classification de sentiment (1 run, 6 textes)

#	Texte (résumé)	Attendu	Résultat Infinix
1	Smartphone excellent	POSITIF	[NÉGATIF] 8% ✗
2	Écran cassé, déçu	NÉGATIF	[NEUTRE] 0% ✗ — justification contradictoire (« le ton du message est négatif, avec des mots comme “déçu” ») pour un label NEUTRE
3	Fonctionne, rien à signaler	NEUTRE	[NÉGATIF] -0,5% ✗ — score de confiance négatif, aberrant
4	Livreur à l’heure, colis en bon état	POSITIF	[NÉGATIF] 80% ✗ — justification pourtant positive (« sans problème », « traité avec prudence »), contradiction interne franche
5	« Service rapide »... 3 semaines (ironie)	NÉGATIF	[NÉGATIF] 90% ✓
6	Correspond à la description, sans plus	NEUTRE	[NÉGATIF], sans justification (génération très courte) ✗

Bilan Partie C : 1/6 — seul le cas piège de l’ironie (#5) est correctement classé. Effondrement net par rapport au S26 Ultra (12/12 sur 2 runs) et à l’A16 (4/6).

Réserve méthodologique : ce score très faible ne doit vraisemblablement pas être lu comme une propriété du Dimensity 6400 (rien dans l’architecture du SoC n’expliquerait un biais de sentiment), mais comme un artefact probable du même changement de build llama-cli que l’incident #4 — soit un changement de paramètres

de sampling par défaut entre versions, soit un traitement différent du template de prompt en mode classification. Aucun message `--no-conversation is not supported` n'apparaît toutefois en mode classification, ce qui suggère un chemin de code différent de celui du mode chat ; la cause exacte du biais reste donc non confirmée et mériterait un test dédié dans un travail futur plutôt qu'une conclusion définitive.

Partie D — Observations de session

Mesure	Valeur observée	Comparaison
Comportement à la saturation	Erreur explicite et bloquante (request (2117 tokens) exceeds the available context size (2048 tokens)), pas de dégradation silencieuse	Cohérent avec le S26 Ultra
Nombre d'échanges avant saturation du contexte	12 (dès la dernière question de la Partie A)	Plus précoce que le S26 Ultra (~13 échanges, 2226 tokens) — écart probablement dû au rappel intégral de l'historique (incident #4), qui gonfle le nombre de tokens par tour plus vite qu'un rechargement normal
RAM disponible en cours de session	De 2180 Mo (29 % libre) en début de session chat à ~2940 Mo (39 % libre) après la dernière classification, sur 7626 Mo total	Marge RAM nettement plus restreinte que le S26 Ultra, dans la même zone que l'A16 (appareil milieu de gamme)
Temps de rechargement par tour (calculé : total – prefill – decode)	~3,6 à 6,2 s sur les appels de classification	Contre ~1,2 s mesuré sur le S26 Ultra — plausible pour un stockage/CPU plus lent, mais non confirmé comme un effet purement matériel compte tenu de l'incident #4

Synthèse — Infinix Hot 60i 5G

Partie	Résultat
A.1 Factuelles (7)	4✓/1△/2✗
A.2 Raisonnement (5)	0✓/0△/4✗ + 1 non évaluable (saturation contexte)
B. Résumé	Bon (2 points complets, 2 partiels)
C. Classification (6)	1/6 — biais systématique vers NÉGATIF, y compris sur des textes clairement positifs
Saturation contexte	Dès la 12e question (2117 tokens)
Incident technique majeur	--no-conversation is not supported (build llama-cli b9-13e6738, plus récent que sur le S26 Ultra/A16) → rappel intégral de l'historique à chaque tour en mode chat
Vitesse	Prefill ~44-56 tok/s, decode très variable (0,8 à 13,8 tok/s selon la charge de contexte accumulée)

Point le plus important pour la rédaction : ce troisième appareil confirme, sur un troisième vendeur de SoC indépendant (MediaTek, après Qualcomm et Samsung), l'échec quasi systématique du modèle sur le raisonnement multi-étapes — le résultat le plus robuste de ce PIR se trouve donc renforcé plutôt que nuancé par ce run. En revanche, le score factuel légèrement dégradé, la saturation plus précoce et surtout l'effondrement en classification (1/6, biais négatif systématique) ne doivent **pas** être interprétés comme des effets du matériel Infinix : ils sont vraisemblablement confondus par une version de llama-cli plus récente que celle utilisée sur les deux appareils précédents, qui modifie le comportement du programme testé lui-même. C'est une limite de reproductibilité à documenter explicitement (chapitre 3, section 8.3, ou chapitre 4) : cloner le même dépôt à des dates différentes sur des appareils différents introduit une variable logicielle non contrôlée, susceptible de produire des écarts de résultats faussement attribuables au matériel testé.

Appareil : Galaxy A26

Modèle testé : Llama 3.2 1B Q4_K_M (identique aux trois appareils précédents, pour comparabilité directe) **Backend** : llama-cli natif en sous-processus (build b4-0eca4d4) **Environnement** : Termux **SoC** : Exynos 1280 (5nm) — quatrième vendeur de SoC du corpus après Qualcomm (S26 Ultra), Samsung (A16) et MediaTek (Infinix) — bien qu'Exynos soit également Samsung, l'Exynos 1280 est une puce distincte de l'Exynos 1330 de l'A16

Incidents techniques rencontrés et résolus

1. **Repo jamais cloné sur cet appareil avant ce test.** Clonage initial depuis GitHub — llama.cpp était déjà compilé (chapitre 2, modèle stocké directement

à la racine du home, pas sous ~/models/ comme sur l’Infinix), mais prototype-cli/ était absent.

2. **import psutil échoue malgré un pkg install python-psutil a priori réussi.** Même symptôme que sur l’Infinix (décalage entre le paquet python système et l’extension compilée python-psutil, avec ici 45 paquets Termux en attente de mise à jour). pkg upgrade a d’abord échoué une première fois avec un message Abort. sans erreur explicite (probablement un téléchargement de 118 Mo interrompu), résolu par une nouvelle tentative. Une fois l’upgrade complet, import psutil fonctionne normalement.
3. **Deuxième occurrence de l’incompatibilité de flag llama-cli déjà rencontrée sur l’Infinix.** Le build installé sur cet appareil (b4-0eca4d4, différent de celui de l’Infinix mais avec le même symptôme) refuse également le flag --no-conversation et retombe sur son propre mode conversation interne, provoquant le même rappel intégral de l’historique à chaque tour de chat que sur l’Infinix. Cette récurrence sur un deuxième appareil cloné récemment renforce l’hypothèse que l’incident dépend de la date de clonage du dépôt plutôt que d’une particularité isolée de l’Infinix.

Partie A — Questions Q/R (12 questions, 1 run)

A.1 — Factuelles simples

#	Question	Réponse A26	Verdict
1	Capitale de la France	« La capitale de la France est Paris. »	✓ Correcte
2	Continents sur Terre	Liste complète et correcte des 7 continents (Afrique, Asie, Amérique du Sud, Amérique du Nord, Australie, Océanie, Europe)	✓ Correcte
3	Symbole chimique de l’eau	Ne répond pas à la question : répète mot pour mot la réponse précédente sur les continents	✗ Incorrecte (aucune réponse à la question posée)
4	Année Révolution française	Noyée dans un rappel, mais contient « 1789 »	✓ Correcte
5	Auteur « Le Petit Prince »	« La réponse est Victor Hugo. »	✗ Incorrecte (attendu : Antoine de Saint-Exupéry)
6	Plus grande planète	« La réponse est Mercure. »	✗ Incorrecte (Mercure est en réalité la plus petite planète du système solaire ; attendu : Jupiter)
7	Jours année bissextile	« Il y a 365, 366 ou 367 jours, selon le type d’année bissextile. La réponse n’est pas établie. » — puis invente et répond lui-même à une nouvelle question non posée (« Quel est le nom de la planète la plus proche du Soleil ? » → « Vénus », également faux)	✗ Incorrecte (refuse de trancher, n’affirme jamais 366 ; hallucination d’un tour de conversation entier)

Bilan A.1 : 3✓ / 0△ / 4✗ — le score le plus faible observé sur les quatre appareils testés (S26 Ultra et A16 : 5/7, Infinix : 4/7).

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Réponse A26	Verdict
8	Croisement de trains	Calcule correctement les deux temps de trajet séparés (3,2h et 3,7h) mais les additionne au lieu de calculer la vitesse combinée → conclut « 7 heures » (attendu ≈1,71h)	✗ Incorrecte
9	RAM FP16→Q4	Invente une formule (« RAM = Vitesse/Facteur »), calcule d’abord « 6 Go » via « $4 \times 1,5 = 6$ » (incohérent), puis se contredit dans la même réponse et conclut finalement « 2,33 Go » — deux résultats différents dans une seule réponse	✗ Incorrecte (attendu 3,5 Go)
10	Pourquoi decode < prefill en vitesse	Affirme que le decode est plus lent « car il nécessite une connexion internet » — faux et contradictoire avec la prémisse même du prototype (inférence locale hors ligne) ; aucune mention memory-bound/compute-bound	✗ Incorrecte
11	RAM smartphone (calcul)	Posée deux fois par erreur (renvoi accidentel de la même question), deux réponses différentes obtenues : la première conclut à tort « pas assez de RAM » (calcul incohérent, mentionne un « modèle 7B » jamais évoqué dans la question) ; la seconde conclut « oui, suffisant » (bonne conclusion) mais avec une justification tout aussi incohérente (invente « 0,75 Go d’autre part », recite encore le « modèle 7B » hors sujet)	△ Partielle (bonne conclusion finale sur la 2e tentative, mais raisonnement non conforme à l’énoncé dans les deux cas)
12	Hybride vs local	Réponse vague sur les avantages/inconvénients de l’hybride seul, ne compare jamais réellement au 100 % local (n’aborde ni la confidentialité ni le fonctionnement hors-ligne, pourtant les points centraux attendus)	✗ Incorrecte

Bilan A.2 : 0✓ / 1△ / 4✗ — même schéma que sur les trois autres appareils, confirmation supplémentaire de l’échec quasi systématique en raisonnement multi-étapes.

Bilan Partie A (12 questions) : 3✓ / 1△ / 8✗.

Réserve méthodologique : comme pour l’Infinix, le score factuel et la qualité de la classification (voir Partie C) ne sont probablement pas directement comparables aux runs S26 Ultra/A16, en raison du même incident de build llama-cli (incident technique #3). Le score de raisonnement multi-étapes, en revanche, reste comparable et confirme, sur un quatrième appareil et un quatrième vendeur de SoC distinct (Exynos 1280, après Snapdragon, Exynos 1330 et Dimensity), l’échec quasi total du modèle sur ce type de tâche.

Partie B — Résumé de texte (1 run, même texte source que les trois autres appareils)

Point clé attendu	Résultat A26
Quantification ($\div 4$ - $\div 8$, perte qualité limitée)	⚠ Partiel — mentionne la baisse de précision et de qualité, mais sans le facteur $\div 4$ - $\div 8$ ni la nuance « perte limitée »
Frameworks (ARM NEON / NPU)	⚠ Partiel — les deux termes apparaissent, mais avec la même erreur d'attribution que sur l'A16 : ARM NEON attribué à la fois à llama.cpp ET à ML Kit GenAI
Gestion mémoire (memory-mapping)	✓ Complet — citation quasi mot pour mot de « memory-mapping (mmap) »
Conclusion (viabilité 1-3B depuis 2020)	✗ Absent — les 5 points s'arrêtent après la mémoire, sans jamais mentionner la conclusion sur la viabilité des modèles 1-3B ni la date de 2020

Verdict : Partiel (1 point complet, 2 partiels, 1 absent) — comparable au run 3 du S26 Ultra, qui avait exactement le même profil.

Métriques : 428 tokens prompt, 250 tokens générés, prefill 7,65 s (56,0 tok/s), decode 27,20 s (9,2 tok/s), total 42,83 s, +1316 Mo RAM (pic sous-processus).

Partie C — Classification de sentiment (1 run, 6 textes)

#	Texte (résumé)	Attendu	Résultat A26
1	Smartphone excellent	POSITIF	[POSITIF] 95% ✓
2	Écran cassé, déçu	NÉGATIF	[NÉGATIF] -80% (score de confiance aberrant, négatif, mais label juste) ✓
3	Fonctionne, rien à signaler	NEUTRE	[NÉGATIF] 0% — justification dit pourtant « l'expression est neutre » ✗ (contradiction interne flagrante)
4	Livreur à l'heure, colis en bon état	POSITIF	[NEUTRE] 100% — justification dit pourtant « un aspect très positive » ✗ (contradiction interne, et mauvais label)
5	« Service rapide »... 3 semaines (ironie)	NÉGATIF	[NÉGATIF] -0,9% ✓
6	Correspond à la description, sans plus	NEUTRE	[NÉGATIF], sans justification (génération très courte) ✗

Bilan Partie C : 3/6 — entre le S26 Ultra (12/12 sur 2 runs) et l'Infinix (1/6), proche de l'A16 (4/6).

Réserve méthodologique : contrairement à l'Infinix (biais systématique vers un seul label), le mode d'échec ici est une **contradiction directe entre le raisonnement écrit et le label produit** sur deux cas (#3 et #4) — le modèle semble « savoir » la bonne réponse dans sa justification mais produire un label différent. Les scores de confiance aberrants et négatifs (-80 %, -0,9 %) réapparaissent comme sur l'Infinix

(−0,5 %), confirmant qu’il ne s’agit pas d’un artefact isolé à un seul appareil, mais potentiellement d’un défaut plus général de formatage de sortie du modèle ou du prototype sur cette classe de build `llama-cli`.

Partie D — Observations de session

Mesure	Valeur observée	Comparaison
Comportement à la saturation	Aucune saturation atteinte sur cette session (12 questions + résumé + 6 classifications complétés sans erreur bloquante)	Contraste avec l’Infinix, qui avait saturé dès la 12e question malgré le même incident de rappel intégral
RAM disponible en cours de session	De 1935 Mo (36 % libre) en début de session chat à ~2381 Mo (44 % libre) après la dernière classification, sur 5427 Mo total	Marge RAM proportionnellement comparable à l’A16, appareil également milieu/entrée de gamme
Vitesse	Pre-fill ~63-99 tok/s, decode ~10-15 tok/s en usage normal (chat, résumé), avec un cas isolé à 4,6 tok/s en classification (case #4)	Plus rapide que l’A16 en valeur brute malgré un segment comparable, cohérent avec les mesures chapitre 2 (Exynos 1280 légèrement plus rapide qu’Exynos 1330 en decode selon le tableau 2.8)

Synthèse — Galaxy A26

Partie	Résultat
A.1 Factuelles (7)	3✓/0△/4✗
A.2 Raisonnement (5)	0✓/1△/4✗
B. Résumé	Partiel (1 point complet, 2 partiels, 1 absent)
C. Classification (6)	3/6 — contradictions internes raisonnement/label, pas de biais directionnel comme sur l’Infinix
Saturation contexte	Non atteinte sur cette session
Incident technique majeur	--no-conversation is not supported (build <code>llama-cli b4-0eca4d4</code>) → même rappel intégral de l’historique qu’ sur l’Infinix ; deuxième appareil consécutif touché

Point le plus important pour la rédaction : ce quatrième appareil confirme une nouvelle fois, sur un quatrième vendeur de SoC (Exynos 1280, après Snapdragon, Exynos 1330 et Dimensity), l’échec quasi total du modèle en raisonnement multi-étapes (0/5) — un résultat désormais établi de façon très robuste sur l’ensemble du corpus testé à ce stade. Le score factuel plus faible (3/7, le plus bas observé) et le mode de dégradation en classification (contradictions internes plutôt que biais directionnel) sont vraisemblablement liés, au moins en partie, au même incident de build `llama-cli` que sur l’Infinix — désormais observé sur deux appareils consécutifs, ce qui en fait un phénomène récurrent plutôt qu’une anomalie isolée. Contrairement à l’Infinix, aucune

saturation du contexte n'a été observée ici, ce qui suggère que l'effet du rappel intégral de l'historique sur la croissance du contexte n'est pas strictement systématique d'un appareil à l'autre (possiblement lié à la longueur variable des réponses générées).

Appareil : Galaxy A71

Modèle testé : Llama 3.2 1B Q4_K_M (identique aux quatre appareils précédents, pour comparabilité directe) **Backend** : llama-cli natif en sous-processus (build b1-b820cc8)

Environnement : Termux **SoC** : Snapdragon 730 (8nm) — déjà utilisé au chapitre 2 pour les benchmarks automatisés (llama.cpp déjà compilé sur cet appareil avant ce test)

Incidents techniques rencontrés et résolus

1. **prototype-cli/ absent** (seul llama.cpp était déjà présent, utilisé pour le chapitre 2) : clonage du dépôt llm-smartphone.
 2. **psutil** : aucun incident cette fois — l'import a fonctionné du premier coup après clonage, sans nécessiter `pkg install python-psutil` ni `pkg upgrade`, contrairement à l'Infinix et l'A26.
 3. **Troisième occurrence de l'incompatibilité de flag llama-cli** déjà rencontrée sur l'Infinix et l'A26. Le build installé sur cet appareil (b1-b820cc8, distinct des deux précédents mais avec le même symptôme exact) refuse le flag `--no-conversation` et retombe sur son propre mode conversation interne, provoquant le même rappel intégral de l'historique à chaque tour. Cette troisième occurrence consécutive, sur un appareil dont llama.cpp avait pourtant déjà été compilé pour le chapitre 2 (donc potentiellement plus tôt que sur l'Infinix/A26), suggère que l'incident touche une plage de versions plus large qu'initialement supposé, plutôt qu'une seule fenêtre de clonage récente.
-

Partie A — Questions Q/R (12 questions, 1 run)**A.1 — Factuelles simples**

#	Question	Réponse A71	Verdict
1	Capitale de la France	« La réponse est : Paris. »	✓ Correcte
2	Continents sur Terre	Aucune réponse à ce tour — répète uniquement la réponse précédente (« La capitale de la France est Paris. »)	✗ Incorrecte (aucune réponse à la question posée)
3	Symbole chimique de l'eau	Aucune réponse à ce tour — fournit en fait, avec un tour de retard, la réponse à la Q2 (liste de 7 « continents », dont un élément erroné : « Pacifique » à la place d'un continent réel)	✗ Incorrecte (aucune réponse à la question posée)
4	Année Révolution française	« La réponse est : 1789. »	✓ Correcte
5	Auteur « Le Petit Prince »	« Antoine de Saint-Exupéry » (répond aussi, avec un tour de retard, la Q3 : « Le symbole chimique de l'eau est H2O. »)	✓ Correcte
6	Plus grande planète	« La plus grande planète du système solaire est Jupiter. »	✓ Correcte
7	Jours année bissextile	Hallucine un tour de conversation utilisateur fictif affirmant « 365,24 jours », puis le confirme lui-même	✗ Incorrecte (n'affirme jamais 366 ; hallucination d'un tour de conversation)

Bilan A.1 : 4✓ / 0△ / 3✗ — comparable à l'Infinix (4/7), meilleur que l'A26 (3/7). À noter : les réponses arrivent parfois décalées d'un tour (la réponse affichée pour une question correspond en réalité à la question précédente), un symptôme cohérent avec l'incident technique #3.

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Réponse A71	Verdict
8	Croisement de trains	Divague (« ils se croisent à la même vitesse car ils sont partants à la même heure »), affirme ensuite « après 1 heure » sans aucun calcul montré, puis demande des informations supplémentaires	✗ Incorrecte
9	RAM FP16→Q4	Conclut « environ 1,5 Go » après un raisonnement incohérent	✗ Incorrecte (attendu ≈3,5 Go)
10	Pourquoi decode < prefill en vitesse	« il est généralement plus lent car le processus implique l'analyse de la structure syntaxique et neurale du modèle » — aucune mention memory-bound/compute-bound	✗ Incorrecte
11	RAM smartphone (calcul)	« Oui, il reste assez de RAM » — bonne conclusion finale, mais aucun calcul ni justification montrés, alors que la question le demande explicitement	△ Partielle
12	Hybride vs local	Argumentation vague des deux côtés, ne cite explicitement ni la confidentialité/le fonctionnement hors-ligne (local) ni la latence réseau (hybride)	✗ Incorrecte

Bilan A.2 : 0✓ / 1△ / 4✗ — même schéma que sur les quatre autres appareils, confirmation supplémentaire de l'échec quasi systématique en raisonnement multi-étapes.

Bilan Partie A (12 questions) : 4✓ / 1△ / 7✗.

Partie B — Résumé de texte (2 runs, même texte source que les quatre autres appareils)

Un premier run, effectué après les douze questions précédentes dans la même session (donc avec un historique déjà chargé), a produit un résumé nettement plus faible que le second — un run de contrôle, lancé en tout début d'une session fraîche avec le même texte, a été effectué pour vérifier la reproductibilité du résultat. Seul ce second run, méthodologiquement plus propre (sans historique accumulé avant le résumé), est retenu comme résultat de référence pour cet appareil.

Point clé attendu	Résultat A71
Quantification (÷4-÷8, perte qualité limitée)	△ Partiel — nomme la réduction de précision numérique des poids, mais sans le facteur ÷4-÷8 ni la nuance de perte de qualité limitée
Frameworks (ARM NEON / NPU)	△ Partiel — cite explicitement ARM NEON, mais omet le NPU
Gestion mémoire (memory-mapping)	✓ Complet — cite « mmap » explicitement et décrit correctement le principe (charger sans dupliquer en RAM)
Conclusion (viabilité 1-3B depuis 2020)	✓ Complet — reprend quasi mot pour mot « 1 à 3 milliards de paramètres... depuis 2020 »

Verdict : Bon (2 points complets, 2 partiels).

Métriques : prefill 41,2 tok/s, decode 7,6 tok/s, total 44,6 s, 1711 Mo RAM (pic sous-processus).

Partie C — Classification de sentiment (1 run, 6 textes)

#	Texte (résumé)	Attendu	Résultat A71
1	Smartphone excellent	POSITIF	[NÉGATIF] -0,9% (confiance 74%) — justification dit pourtant « appréciation positive » ✗ (contradiction interne)
2	Écran cassé, déçu	NÉGATIF	[NÉGATIF] -99% ✓
3	Fonctionne, rien à signaler	NEUTRE	[NÉGATIF] 90% — justification dit pourtant « neutre et positive » ✗ (contradiction interne)
4	Livreur à l'heure, colis en bon état	POSITIF	[NÉGATIF] 80% — justification dit pourtant « le texte est positif » ✗ (contradiction interne)
5	« Service rapide »... 3 semaines (ironie)	NÉGATIF	[NÉGATIF] 80% — justification cohérente cette fois (mentionne la contradiction attente/« rapide ») ✓
6	Correspond à la description, sans plus	NEUTRE	[NÉGATIF] 0% — justification dit pourtant « neutre » ✗ (contradiction interne)

Bilan Partie C : 2/6.

Réserve méthodologique : les **six textes sur six** sont étiquetés [NÉGATIF], quel que soit leur contenu réel — un biais directionnel total, encore plus marqué que celui déjà observé sur l'Infinix (5/6). Sur quatre des six textes, la justification textuelle contredit ouvertement le label produit (« le texte est positif » → étiqueté NÉGATIF), un mode de dégradation qui combine à la fois le biais directionnel de l'Infinix et les contradictions internes de l'A26. Le succès sur le cas piège de l'ironie (#5) doit être relativisé pour la même raison que sur l'Infinix : un biais généralisé vers NÉGATIF rend ce succès probablement mécanique plutôt qu'une réelle détection de l'ironie.

Partie D — Observations de session

Mesure	Valeur observée	Comparaison
Comportement à la saturation	Aucune saturation atteinte — la session Q/R + résumé (14 entrées journalisées) s’est achevée sans erreur bloquante ; la classification a été effectuée dans une session redémarrée séparément	Contraste avec l’Infinix, qui avait saturé dès la 12e question malgré le même incident de rappel intégral
RAM disponible	De 4166-4278 Mo (55-57 % libre) à 4570 Mo (61 % libre) selon les lancements, sur 7520 Mo total	Marge RAM confortable, comparable aux autres appareils milieu de gamme
Vitesse	Pre-fill ~35-43 tok/s, decode ~6-9 tok/s en usage normal (chat, résumé, classification)	Plus lent en decode que les vitesses mesurées au chapitre 2 pour ce même appareil via llama-bench (~11,4-11,7 tok/s), cohérent avec le surcoût de rechargement par tour déjà documenté sur les autres appareils

Synthèse — Galaxy A71

Partie	Résultat
A.1 Factuelles (7)	4✓/0△/3✗
A.2 Raisonnement (5)	0✓/1△/4✗
B. Résumé	Bon (2 points complets, 2 partiels — run de référence en session fraîche)
C. Classification (6)	2/6 — biais total vers NÉGATIF (6/6), combiné à des contradictions internes justification/label sur 4 cas
Saturation contexte	Non atteinte sur cette session
Incident technique majeur	--no-conversation is not supported (build llama-cli b1-b820cc8) → troisième appareil consécutif touché

Point le plus important pour la rédaction : ce cinquième appareil confirme une nouvelle fois, sur un troisième vendeur de SoC (Qualcomm, après Samsung Exynos et MediaTek), l’échec quasi total du modèle en raisonnement multi-étapes (0/5) — désormais établi sur les cinq appareils testés à ce stade, sans une seule exception. Le score factuel (4/7) et le résumé (Bon, en session fraîche) restent dans la moyenne du corpus, mais le score de classification (2/6, biais total vers NÉGATIF) confirme, pour la troisième fois consécutive, que l’incompatibilité de flag llama-cli dégrade fortement la fiabilité de cette tâche particulière — un phénomène désormais bien établi plutôt qu’une anomalie isolée.

Test 3 — Comparaison avec une solution Google on-device (AI Edge Gallery)

Objectif : comparer le prototype llama.cpp/Llama 3.2 1B (testé ci-dessus) à une solution d'inférence on-device propriétaire concurrente, sur le même jeu de 12 questions de la Partie A du protocole (protocole_validation_chatbot.md).

Note méthodologique importante — substitution de modèle : le mémoire (chapitres 1 à 3) référence initialement “Gemma 2 2B” comme baseline de comparaison Google on-device. Au moment de ce test (application Google AI Edge Gallery, août 2026), **Gemma 2 2B n'est plus proposé au téléchargement dans le catalogue de l'application** — seuls des modèles plus récents sont disponibles (Gemma-4-E4B-it, Gemma-3n-E2B-it, Gemma-3n-E4B-it, Gemma3-1B-IT, et d'autres non listés ici). Il s'agit d'une évolution du catalogue de l'application postérieure à la rédaction initiale du mémoire, pas d'un choix arbitraire. **Gemma3-1B-IT (584 Mo) a été retenu en remplacement**, car c'est le modèle de ce catalogue le plus proche en nombre de paramètres (1B) de Llama 3.2 1B déjà testé — préservant une comparaison à échelle de paramètres comparable plutôt que de comparer un modèle 1B à un modèle 4B, ce qui aurait faussé la comparaison en faveur de Google pour de mauvaises raisons (plus de paramètres, pas une meilleure solution technique).

Modèle testé : Gemma3-1B-IT (584,4 Mo) **Application** : Google AI Edge Gallery, interface “AI Chat” **Appareil** : Galaxy S26 Ultra (même appareil que les runs 1-3 ci-dessus) **Méthode** : les 12 questions de la Partie A posées une par une, dans l'ordre, via l'interface de chat de l'application ; latence par réponse relevée directement dans l'interface.

A.1 — Factuelles simples

#	Question	Réponse Gemma3-1B-IT	Verdict	Latence
1	Capitale de la France	“La capitale de la France est Paris.”	✓ Correcte	462 ms
2	Continents sur Terre	“Il y en a six : Afrique, Europe, Asie, Amérique du Nord, Amérique du Sud, Océanie”	✓ Correcte (convention à 6 continents acceptée par le protocole)	1,2 s
3	Symbole chimique de l’eau	“H2O”	✓ Correcte	330 ms
4	Année Révolution française	“La Révolution française est terminée en 1789.”	✓ Correcte (année exacte ; formulation maladroite, “terminée” au lieu de “a eu lieu”)	337 ms
5	Auteur “Le Petit Prince”	“Antoine de Saint-Exupéry”	✓ Correcte	399 ms
6	Plus grande planète	“Neptune”	✗ Incorrecte (attendu Jupiter)	286 ms
7	Jours année bissextile	“365 jours”	✗ Incorrecte (attendu 366)	531 ms

Bilan A.1 Gemma3-1B-IT : 5✓ / 0⚠ / 2✗ sur 7 — échoue sur les deux mêmes catégories de question (planète, jours bissextile) que Llama 3.2 1B avait également ratées lors de certains runs, mais avec des erreurs différentes dans le détail.

A.2 — Raisonnement multi-étapes

#	Question (résumée)	Réponse Gemma3-1B-IT	Verdict	Latence
8	Croisement de trains	“Après environ 15 minutes pour le premier train et environ 20 minutes pour le second train.” — ne calcule pas de vitesse combinée, produit deux durées disjointes sans rapport avec la question posée	✗ Incorrecte	518 ms
9	RAM FP16→Q4	“Il faudrait environ 68 Go de RAM” — réponse incohérente, sens de la quantification (réduction, pas augmentation) mal compris	✗ Incorrecte	574 ms
10	Pourquoi decode < prefill en vitesse	Explication vague, n’établit pas correctement la distinction memory-bound (decode) / compute-bound (prefill)	✗ Incorrecte	1,3 s
11	RAM smartphone (calcul)	Invente un “minimum de 14 Go de RAM requis”, n’effectue pas le calcul 8–3–1,5–0,5 demandé, ne conclut pas par oui/non	✗ Incorrecte	1,4 s
12	Hybride vs local	Premier segment de réponse confus et auto-contradictoire, ne mentionne ni confidentialité ni fonctionnement hors-ligne pour le local	✗ Incorrecte	1,3 s

Bilan A.2 Gemma3-1B-IT : 0✓ / 0△ / 5✗ sur 5 — échec total, comparable ou légèrement en retrait par rapport à Llama 3.2 1B (0-1✓ / 0-1△ / 4-5✗ selon les runs 1-3).

Bilan Partie A (12 questions) Gemma3-1B-IT : 5✓ / 0△ / 7✗.

Comparaison synthétique avec Llama 3.2 1B (llama.cpp)

Partie	Llama 3.2 1B (llama.cpp)	Gemma3-1B-IT (AI Edge Gallery)
A.1 Factuelles (7)	Run 1 : 5✓/1△/1✗ — Run 2 : 5✓/0△/2✗	5✓/0△/2✗
A.2 Raisonnement (5)	Run 1 : 0✓/1△/4✗ — Run 2 : 0✓/1△/4✗ — Run 3 : 1✓/0△/4✗	0✓/0△/5✗
Latence par réponse	Non directement comparable (total_time_s inclut prefill + decode + rechargement du sous-processus, ~4,5 à 9,5s sur A.2 au run 3)	286 ms - 1,4 s (inférence NPU intégrée à l'application, pas de rechargement de sous-processus entre tours)

Analyse : sur les questions de raisonnement multi-étapes (A.2), les deux modèles échouent massivement (Gemma3-1B-IT : 0/5 ; Llama 3.2 1B : 0-1/5 selon le run) — ce résultat, loin d'affaiblir la conclusion déjà établie dans les chapitres 3 et 4 sur la limite de raisonnement des petits modèles embarqués (~1B paramètres), la renforce : elle n'est pas spécifique à llama.cpp ni à un défaut d'implémentation du prototype, mais bien une limite du modèle et de son échelle de paramètres, observée de façon cohérente sur deux frameworks d'inférence indépendants (llama.cpp/CPU vs solution propriétaire Google/NPU). Sur les questions factuelles simples (A.1), les deux modèles échouent également sur des connaissances générales basiques, bien qu'avec des erreurs différentes en surface. Les latences ne sont pas directement comparables comme mesure de performance brute : celles de Gemma3-1B-IT reflètent uniquement le temps de génération dans l'application, alors que total_time_s du prototype inclut, en plus du calcul, le rechargement complet du sous-processus llama-cli à chaque tour (~1,2s fixe, voir mesure dédiée au temps de rechargement ci-dessus) — une charge que l'architecture d'AI Edge Gallery n'a pas, puisqu'elle garde le modèle chargé en mémoire entre les tours.

Conclusion pour la rédaction : ce test confirme, avec un second modèle de taille comparable (~1B paramètres) et un second framework d'inférence indépendant, que la limite de raisonnement multi-étapes documentée dans ce mémoire n'est pas un artefact du prototype llama.cpp mais une caractéristique partagée par les modèles de cette échelle de paramètres, quelle que soit la solution technique utilisée pour les exécuter.

Test 4 — Mesure de la latence de llama.cpp sur le Galaxy S26 Ultra (20/09/2026)

Objectif : remplacer par une mesure la valeur « ~1,5 à 2 s » qui figurait dans le chapitre 2 (tableaux 2.10 et 2.12) et dans le chapitre 4 (section 4.1.7) pour la latence d’une réponse courte de llama.cpp sur le Galaxy S26 Ultra. Cette valeur n’était rattachée à aucune trace : la sortie de benchmark_complet.sh n’avait pas été conservée (aucun dossier ~/benchmark_results/ sur l’appareil) et le seul fichier retrouvé, metrics.json, ne contenait que des entrées du 30/08/2026 (la copie archivée ci-dessous ne conserve que les entrées du 20/09/2026).

Conditions du test

Élément	Valeur
Appareil	Galaxy S26 Ultra (Snapdragon 8 Elite, 12 Go RAM)
Environnement	Termux natif
Outil	chatbot.py (prototype du chapitre 3), qui pilote le binaire llama-cli en sous-processus
Build llama-cli	b10154-0e4a03622
Modèle	Llama 3.2 1B Instruct Q4_K_M (~/models/llama-3.2-1b-instruct-q4_k_m.gguf)
Prompt	Le même que pour le test LiteRT / AI Edge Gallery : « Explique-moi le concept d’intelligence artificielle en 3 phrases. »
Protocole	3 runs, chacun dans une session neuve (contexte vide)

Résultats bruts (3 runs, entrées 4 à 6 de metrics.json)

Run	Latence totale	dont chargement du modèle	Decode	Prefill	Tokens générés
1 (06:37:24)	3,14 s	1,04 s	55,32 tok/s	216,16 tok/s	77
2 (06:37:41)	3,35 s	1,04 s	55,45 tok/s	218,16 tok/s	89
3 (06:37:58)	2,72 s	1,04 s	55,37 tok/s	217,98 tok/s	54
Moyenne	3,07 s (2,72 à 3,35 s)	1,04 s	55,38 tok/s	217,43 tok/s	

Le prompt fait 124 tokens dans les trois runs. Le chargement du modèle représente environ 1,04 s par réponse (le prototype recharge le modèle à chaque tour, voir section 3.2.1 du chapitre 3) ; la latence hors rechargement est donc d’environ 2,03 s. Consommation mémoire relevée : environ 1680 Mo. La latence totale dépend du nombre de tokens générés (54 à 89 selon le run) ; le débit de décodage, lui, est stable à $\pm 0,1$ tok/s.

Cohérence avec les autres entrées du fichier : les trois premières entrées (06:35 à 06:37) sont des essais préliminaires. L’entrée 1 utilise le même prompt de 124 tokens (58 tokens générés, 3,01 s au total, décodage à 55,76 tok/s) et confirme l’ordre de grandeur. Les entrées 2 et 3 correspondent à d’autres tâches du prototype (prompts

de 163 et 594 tokens, réponses de 27 et 18 tokens) : elles ne sont pas comparables (5,76 s au total pour l'entrée 3, dominé par un prefill de 4,2 s sur 594 tokens à 141 tok/s).

Conclusion : la latence d'une réponse courte de llama.cpp sur le S26 Ultra est de **3,07 s en moyenne avec rechargement du modèle** (arrondie à « 3,1 s » dans le mémoire) et d'environ **2,0 s hors rechargement**. La valeur « ~1,5 à 2 s » n'était pas étayée et a été retirée. Conséquences appliquées dans le mémoire : tableaux 2.10 et 2.12 (chapitre 2), paragraphe « Protocole de la mesure llama.cpp » (section 3.4 du chapitre 2), section 4.1.7 du chapitre 4 (LiteRT environ 2 à 3 fois plus lent en latence totale au lieu de 4 fois), et paragraphe de test complémentaire en section 3.8 du chapitre 3.

Réserve : ce test compare deux modèles de tailles différentes (Llama 3.2 1B pour llama.cpp, Gemma 4 E2B d'environ 2B paramètres pour LiteRT) ; l'écart de latence ne peut donc pas être attribué au seul framework.

Trace : le fichier metrics.json du S26 Ultra (chemin sur l'appareil : ~/llm-smartphone/prototype-cli/results/metrics.json) est archivé dans results/metrics_-s26_2026-09-20.json à la racine du dépôt. Il contient six entrées, toutes du 20/09/2026 : les trois runs du test en sont les entrées 4 à 6.